

Google Stock Price Forecasting

From Statistical Time Series to Machine Learning — A Complete Forecasting Pipeline

Author: Dev | M.Sc. Statistics, University of Delhi

Dataset: GOOGL Daily Stock Data (Yahoo Finance) | March 2024 – March 2026

Models: SARIMAX → Random Forest | Gradient Boosting | XGBoost

Project Date: March 2026

A non-technical summary of methodology, findings, and strategic recommendations for business stakeholders.

1. What Problem Were We Solving?

Stock prices move every second, driven by earnings, news, economic shifts, and market sentiment. For investors, traders, and financial analysts, the ability to forecast where a stock is likely to go — even one or two days ahead — is enormously valuable.

This project built a complete price forecasting pipeline for Google (GOOGL) stock using two years of daily trading data (March 2024 – March 2026). We started with classical statistical methods, identified their limits, then moved to machine learning — documenting exactly what worked, what failed, and why at every step.

What	Detail
Data source	Yahoo Finance — official market data
Period covered	March 13, 2024 to March 13, 2026 (2 years)
Trading days	502 cleaned observations
Data columns	Date, Opening Price, Closing Price, High, Low, Volume
Goal	Forecast the daily closing price of GOOGL stock

2. Data Preparation — What We Cleaned and Why

The raw data file from Yahoo Finance arrived with formatting issues that needed to be fixed before any analysis could begin:

- **Two junk header rows** were removed — Yahoo Finance includes a ticker label row and a blank row above the actual column headers that confuse automated tools.
- **A mislabelled column** called 'Price' was renamed to 'Date' — its actual content.
- **Data types were corrected:** dates were set to proper date format, prices to decimal numbers, and trading volume to whole numbers.
- **No data was deleted** for statistical reasons. Price spikes and volume surges were verified as genuine market events (earnings releases, major announcements) and kept — removing real events would harm model accuracy.

Key decision: We deliberately kept all outliers. Unusual price or volume days correspond to real-world events that carry predictive information — removing them would make the model less accurate, not more.

3. What the Data Revealed — Key Patterns

Before building any model, we analysed the data visually and statistically to understand its structure. Three important patterns emerged:

Pattern 1: Two Distinct Price Regimes

Google’s stock moved through two clearly separate phases over the two years:

Period	Price Range	Regime
FY 2024–25 (Mar 2024 – Mar 2025)	£138 – £220	Lower regime
FY 2025–26 (Mar 2025 – Mar 2026)	£220 – £343 (peak)	Higher regime

This two-regime structure turned out to be the single most important feature of the dataset — it caused the first machine learning models to fail completely (explained in Section 5).

Pattern 2: Consistent Seasonal Cycle

Prices consistently rise from April through January each year, then pull back in February–March. This intra-year rhythm justified testing models that explicitly capture seasonal patterns.

Pattern 3: Volume Does Not Directly Predict Price

Statistical tests (Granger causality) showed that trading volume has no direct linear relationship with future price movements. However, volume was kept in the machine learning phase — because ML models can detect complex non-linear patterns that these tests cannot.

4. The Statistical Model — SARIMAX

The first modelling approach used SARIMAX — a classical statistical method designed specifically for time series data. It captures two types of patterns simultaneously: a trend component (how prices drift over time) and a seasonal component (how prices repeat cycles within each period).

How We Selected the Best Model

Rather than manually guessing model settings, we ran an automated search across dozens of configurations — testing different seasonal periods (1 week, 1 trading month, 2 months) and comparing results. The search was run twice independently to confirm the winner was stable.

The winning configuration: a seasonal period of 20 trading days — exactly one trading month. This captured the intra-month cyclical pattern in Google’s stock that a simple model completely missed.

Performance of the Statistical Model

Metric	Result	What it means
RMSE (error in price terms)	\$22.87	On average, predictions were ~\$23 off from the actual price
MAPE (percentage error)	6.70%	Predictions were off by about 6.7% on average
Seasonal period	20 days	Captures one full trading month cycle

A MAPE of 6.70% is a meaningful improvement over the non-seasonal model (16.5% error) — confirming that the seasonal component was genuinely capturing real patterns in the data. This became the benchmark the machine learning models had to beat.

One attempt to further improve the model using a mathematical transformation (Box-Cox) was abandoned after a pipeline error caused results to worsen dramatically. The original model remained the best traditional approach.

5. The Critical Discovery — Why the First ML Models Failed

The first machine learning models — Random Forest and XGBoost — were trained on raw price data. The results were alarming:

Model	Test R ² Score	Verdict
Random Forest	-10.13	Complete failure
XGBoost	-5.75	Complete failure
Predicting mean	0.00	Even this beat the models

A negative R² score means the model performed worse than simply predicting the average price every single day — a completely useless result. No amount of tuning would fix this.

Root Cause: The Regime Mismatch Problem

The investigation revealed a fundamental issue:

Data Split	Price Range	Regime
Training data (what the model learned from)	\$144 – \$269	Lower regime
Test data (what the model was asked to predict)	\$267 – \$343	Higher regime — never seen before

Machine learning tree models are interpolators — they can only predict values within the range they were trained on. The model had never seen a price above \$269 during training, yet was asked to predict prices of \$280–\$340. It had no basis for this and defaulted to predicting \$190–\$220 (the middle of what it knew) while actual values were \$280–\$340.

This is not a flaw in the specific model — it is a fundamental property of ALL tree-based machine learning models. The fix required rethinking what the model should predict, not which model to use.

6. The Fix — Predicting Returns Instead of Prices

The solution was elegant: instead of predicting the absolute closing price (e.g. “\$310”), train the model to predict the daily percentage change (e.g. “+1.2% today”).

Why Returns Solve the Problem

Daily percentage returns stay in a consistent range regardless of the absolute price level:

Price Level	Typical Daily Return Range	Is this consistent?
\$170 (lower regime)	-3% to +3%	Yes
\$320 (higher regime)	-3% to +3%	Yes — same range

The model no longer needed to extrapolate to unseen price levels. It only needed to learn patterns in daily percentage movements — which are consistent across both price regimes.

#	Approach	Test R^2	Outcome
1	Raw price features	-10.13	Complete failure
2	Simple return features	+0.16	Positive but overfit
3	Full return feature engineering	+0.849	Strong result ✓

The entire improvement from $R^2 = -10$ to $R^2 = +0.849$ was achieved through understanding why the model was failing and redesigning the features — not by switching to a different algorithm or tuning more parameters.

7. What Information Did We Feed the Models?

Twenty features were engineered from the raw OHLCV data. All are expressed as percentage changes or ratios, making them scale-independent and regime-agnostic. Here is what each group captures:

Feature Group	What it captures in plain English
Daily Returns (Open, High, Low, Close, Volume)	How much each price measure moved today compared to yesterday — the basic building blocks of all other features.

Intraday Return	How much the stock moved during the trading session itself (from open to close) — captures intraday market sentiment.
Gap Return	The overnight price jump between yesterday's close and today's open — captures the impact of after-hours news.
Multi-Period Returns (2-day, 5-day, 10-day)	How much the stock has moved over the past week or fortnight — captures short and medium-term momentum.
Lagged Returns (yesterday, 2 days ago, 3 days ago)	Recent history of the stock's own movements — allows the model to learn from its own recent past.
Rolling Volatility (2-day, 5-day, 10-day)	How much the stock has been jumping around recently — periods of high volatility often signal trend changes.
Volatility Ratios	Whether volatility is rising or falling relative to longer periods — an early warning signal for market instability.
Lagged Exogenous Variables (High lag 6, Open lag 7)	The High and Open prices from 6 and 7 trading days ago — identified through statistical causality tests as having genuine predictive power.

After feature importance analysis, 3 low-contribution features were removed. The final model used 17 features.

8. Machine Learning Results — Four Models Compared

Four models were trained and tuned using time-series-aware cross-validation (walk-forward testing, not random splits — to prevent the model from ‘cheating’ by peeking at future data during training).

Model	Test R ²	Test Error (RMSE)	Overfit Gap	Verdict
Random Forest ★ Winner	84.89%	0.00544	0.132 (smallest)	Best generalization
Voting Regressor (ensemble)	83.75%	0.00564	0.158	Strong, near winner
Gradient Boosting	81.98%	0.00594	0.178	Good but more overfit
XGBoost	79.77%	0.00629	0.201 (largest)	Most overfit

R² of 84.89% means the Random Forest model explains 85% of the variation in daily return movements on data it had never seen during training. The remaining 15% reflects genuine market unpredictability — news, sentiment, macro events — that cannot be predicted from historical price patterns alone.

Why Random Forest Won

- **Best test accuracy (84.89% R²)** — highest generalization to unseen data.
- **Smallest overfitting gap (0.132)** — train and test performance stayed closest together. The model learned genuine patterns, not memorized noise.
- **XGBoost memorised training data** (99.84% train R² vs 79.77% test R²) despite its built-in regularization — a 20-point collapse between training and testing is a clear signal of overfitting.

9. Statistical Model vs Machine Learning — How Do They Compare?

The two approaches cannot be compared directly on a single number because they predict different things (price vs percentage return). The meaningful comparison is qualitative:

	SARIMAX (Statistical)	Random Forest (ML)
What it predicts	Raw closing price in dollars	Daily percentage return
Average error	~\$23 per day (MAPE: 6.7%)	0.54% return error
Handles two price regimes	Partially	Yes — via return transformation
Captures seasonality	Yes — explicit 20-day cycle	Indirectly via lag features
Residual pattern remaining	Yes (visible in diagnostics)	Minimal
Interpretability	High — explicit equations	Moderate — feature importance
Best use case	When seasonality is dominant	When patterns are complex & non-linear

10. Three Business Recommendations

1 Use Return-Based ML Models for Short-Term Daily Forecasting

The Random Forest model achieving 84.89% R² on daily return prediction is a strong result for financial forecasting. For any operational use case requiring short-term directional signals — portfolio rebalancing triggers, trading alerts, or risk monitoring — a return-based ML model is significantly more reliable than a price-based model. The key insight: predict the percentage move, not the absolute price. This applies to any stock with a meaningful price regime shift in its history.

Strategic Impact: *More reliable daily signals; avoids the regime extrapolation failure that plagued the initial models.*

2 Enrich the Model With External Signals Before Production Deployment

The current model uses only the stock's own historical price and volume data. In practice, Google's stock is heavily influenced by factors not visible in OHLCV data: earnings announcements, AI sector

news, macro interest rate decisions, and competitor movements. Integrating even one or two external signals — a sector ETF return, a news sentiment score, or a macro indicator — could meaningfully close the remaining 15% of unexplained variation. The existing feature engineering pipeline is designed to accept these additions without restructuring.

Strategic Impact: Reduces the ‘unexplained’ component; makes predictions more robust to macro-driven moves.

3 Never Deploy a Tree-Based Model Trained Only on a Historical Price Range

The most important lesson of this entire project has direct operational implications: any tree-based ML model trained on historical stock data will fail when the stock enters a price range it has never seen during training. This is not a model flaw — it is a fundamental architectural property. The fix is always the same: train on returns, not prices. Any team deploying ML for financial forecasting should treat this as a non-negotiable requirement, not an optional refinement. Retraining models regularly (e.g. monthly rolling windows) also helps prevent the training distribution from drifting too far from current market conditions.

Strategic Impact: Prevents catastrophic prediction failures when market conditions shift — protects downstream decision quality.

11. Interesting Things We Found Along the Way

A model can be 100% confident and completely wrong.

When high trading volume days were examined, models sometimes predicted with extreme certainty — yet got the direction wrong. In financial markets, high confidence in a prediction should raise caution, not lower it.

Seasonality in stock data is real and measurable.

Google’s stock consistently follows an intra-year cycle — rising through most of the year and pulling back in February–March. Statistical models that ignore this cycle perform roughly 2.5x worse than those that account for it.

Volume doesn’t predict price directly — but it does contribute non-linearly.

Standard statistical tests showed trading volume has no direct linear effect on future prices. Yet the ML model found it useful — because volume interacts with price movements in complex ways that only non-linear models can detect.

Diagnosing why a model fails is more valuable than trying another model.

The journey from $R^2 = -10$ to $R^2 = +0.85$ was not achieved by switching algorithms — it was achieved by understanding the root cause of failure (regime mismatch) and redesigning features to address it specifically.

Walk-forward validation is essential for time series.

Standard ML cross-validation randomly splits data, allowing the model to ‘learn from the future’ during training. For any time series task, this produces falsely optimistic scores. Proper walk-forward validation showed more conservative — and honest — estimates of real-world performance.

12. Current Limitations & Next Steps

Known Limitations

- **No external signals:** The model only sees Google's own price history. News sentiment, macro data, sector performance, and competitor movements are not included.
- **Compounding drift over long horizons:** Predicting percentage returns and converting back to prices works well for 1–2 day forecasts but introduces accumulating error over 40+ days. Small daily prediction errors compound into visible price gaps.
- **Single test window:** The 90/10 split gives one evaluation window. Rolling window evaluation across multiple time periods would give a more robust picture of real-world reliability.

Recommended Next Steps

- **Add external signals:** Integrate a sector ETF return (e.g. QQQ), a news sentiment API, or a macro indicator (e.g. VIX) to capture drivers not visible in OHLCV data.
- **Rolling retraining:** Retrain the model monthly on the most recent 12–18 months of data to prevent distribution drift as market conditions evolve.
- **Extend to other stocks:** The return-based feature engineering pipeline and regime-handling methodology developed here is generic — it can be applied to any equity with minimal modification.